SAML · Documentación técnica

Sistema Automatizado de Monitoreo de Licitaciones para GRECO
Versión 1.0 (versión final) · Rama `main` · Noviembre de 2026
Proyecto SI4008 · Universidad EAFIT

Este documento es para quien instale, opere en lo técnico o mantenga SAML. El manual para el equipo comercial de GRECO es `Manual_Usuario_SAML.pdf`, en esta misma carpeta.

---

1. Qué hace el sistema

SAML consulta las licitaciones **adjudicadas** de Licitaciones.info, las limpia, decide a qué asesor comercial de GRECO le corresponde cada obra y la deja lista en el archivo Excel de ese asesor, con las mismas 71 columnas, fórmulas y formatos que GRECO ya usa.

```
Licitaciones.info (API)
        │  1. Ingesta: CSV por ejecución
        ▼
Limpieza ─► Reparto por asesor ─► (LLM para las entidades sin regla)
        │
        ▼
datos/salida/SAML_<fecha>/
   ├─ <maestro>_actualizado_<fecha>.xlsx   copia del maestro con las obras insertadas (con Excel)
   ├─ Nuevas_<maestro>_<fecha>.xlsx        obras nuevas de cada asesor (respaldo)
   └─ SAML_revision_<fecha>.xlsx           resumen, descartes, incidencias
```

Los **4 Excel maestros** de GRECO (uno por asesor) son la fuente de verdad. SAML los lee y **nunca los modifica**: escribe copias nuevas en la carpeta de salida.

Maestro	Asesor	Hojas de obras
`Obras_IDU_Gustavo_VF1.xlsx`	Gustavo	IDU
`Obras_Acueducto_Alcaldias_Mariana_VF1.xlsx`	Mariana	Acueducto, Alcaldías Locales, Megaproyectos (ref.)
`Obras_IDRD_Privadas_Municipios_Asesor3_VF1.xlsx`	Asesor 3 (por contratar)	IDRD, Obras Privadas, Municipios Aledaños
`Obras_Sugeine_VF1.xlsx`	Sugeine	IDU, Alcaldías Locales, Acueducto, Municipios Aledaños, IDRD, Obras Privadas, Edificaciones y Mant. (+ Cobertura clientes)

---

2. Arquitectura y componentes

Componente	Archivo	Responsabilidad
Configuración de la ingesta	`Ingesta/config.py`	Lee `Ingesta/.env` (credenciales, país, filtros)
Cliente de la API	`Ingesta/api_client.py`	Login JWT, perfiles, consulta paginada de procesos; reintentos ante fallos de red y 429/502/503/504
Ingesta	`Ingesta/extract.py`	LOGIN → PERFILES → CONSULTAR PROCESOS → `Ingesta/output/licitaciones_<fecha>.csv`
Limpieza	`Transformacion/limpieza_licitaciones.py`	Consolida CSV por `idContrato`, filtra `Adjudicado`, normaliza, extrae ganadores y fechas del cronograma, registra incidencias
Maestros	`Transformacion/maestros.py`	Lee los maestros: hojas, plantilla de 71 columnas, asesor, obras existentes, clientes de Sugeine, listas
Reparto	`Transformacion/obras_asesores.py`	Reglas de reparto, duplicados, Sugeine, armado de cada fila
Clasificación con LLM	`Transformacion/clasificador_llm.py`	OpenAI con salida estructurada para las entidades sin regla
Escritura	`Transformacion/generar_archivos_asesores.py`	Archivos `Nuevas_…` y `SAML_revision_…` con openpyxl
Copias de los maestros	`Transformacion/actualizar_maestros.py`	Inserta las obras en copias de los maestros con Excel (COM, Windows)
Orquestador	`Transformacion/ejecutar_proceso.py`	Punto de entrada: ingesta opcional, generación y verificación
Atajos de Windows	`Ejecutar_SAML.bat`, `Actualizar_maestros.bat`	Doble clic: proceso completo; copias de los maestros a partir de una salida de n8n
n8n	`n8n/`	Instancia local en Docker con dos flujos: proceso completo y demo
Pruebas	`Transformacion/tests/`	33 pruebas con datos y maestros sintéticos

Tecnologías: Python 3.13/3.14, `requests`, `python-dotenv`, `openpyxl`, `openai` (Responses API con salida estructurada y Pydantic), `pywin32` (solo Windows), n8n 2.42.7 en Docker, Microsoft Excel (para las copias de los maestros).

3. Estructura del repositorio

```
SAML/
├── Ejecutar_SAML.bat           # doble clic: proceso completo
├── Actualizar_maestros.bat     # doble clic: copias de los maestros desde la última salida
├── Ingesta/                    # extracción desde Licitaciones.info
│   ├── .env.example            # plantilla de credenciales (el .env real NO se versiona)
│   └── output/                 # CSV de cada ingesta (no se versiona)
├── Transformacion/             # limpieza, reparto, LLM, escritura y verificación
│   ├── .env.example            # plantilla de la clave de OpenAI (el .env real NO se versiona)
│   └── tests/
├── n8n/                        # Dockerfile, docker-compose y los dos flujos
├── datos/                      # NO se versiona
│   ├── maestros/               # los 4 Excel de GRECO (entrada, solo lectura)
│   └── salida/SAML_<fecha>/    # resultados de cada ejecución
└── docs/                       # entregables del proyecto
```

`datos/` y los `.env` están en `.gitignore`: contienen credenciales, NIT, ventas y clientes de GRECO, y el repositorio es público.

4. Instalación

4.1 Equipo Windows (uso completo, recomendado)

Requisitos: Python 3.13 o superior, Git, Microsoft Excel.

```
git clone https://github.com/svillav4/SAML.git
cd SAML
python -m venv .venv
.venv\Scripts\python -m pip install -r Ingesta/requirements.txt -r Transformacion/requirements.txt
copy Ingesta\.env.example Ingesta\.env
copy Transformacion\.env.example Transformacion\.env
mkdir datos\maestros
```

Luego: 1. Escribir las credenciales de Licitaciones.info en `Ingesta/.env` y la clave de OpenAI en `Transformacion/.env` (sección 5). 2. Copiar los 4 maestros de GRECO a `datos/maestros/`.

`pywin32` se instala solo en Windows (marcador `sys_platform == "win32"` en `requirements.txt`).

4.2 n8n en Docker

Requisito: Docker Desktop. Desde la raíz del repositorio:

```
docker --context desktop-linux compose -f n8n/docker-compose.yml up -d --build
docker --context desktop-linux exec saml-n8n n8n import:workflow --
input=/proyecto/n8n/flujo_local_licitaciones.json
docker --context desktop-linux exec saml-n8n n8n import:workflow --
input=/proyecto/n8n/flujo_demo_csv_locales.json
```

En Git Bash de Windows, anteponer `MSYS_NO_PATHCONV=1` para que no cambie las rutas `/proyecto/...`. El editor queda en `http://localhost:5678`; la primera vez pide crear la cuenta de propietario local. Detalles en `n8n/README.md`.

---

5. Configuración

`Ingesta/.env`

Variable	Valor	Uso
`LICITACIONES_EMAIL`, `LICITACIONES_PASSWORD`	Cuenta de Licitaciones.info	Login de la API
`LICITACIONES_COUNTRY`	`colombia`	Subdominio (`col`)
`LICITACIONES_PERFILES`	vacío	Si está vacío, se usan todos los perfiles de la cuenta
`LICITACIONES_ESTADO`	`Adjudicado`	**Obligatorio**: el proceso se detiene si es otro valor
`LICITACIONES_FILTRAR_NUEVOS`	`true` / `false`	`true`: solo procesos que la cuenta no ha visto
`LICITACIONES_KEYWORD`, `LICITACIONES_QUERY_EXTRA`	opcional	Filtros adicionales
`LICITACIONES_PAGE_LIMIT`	`30`	Registros por página (tope de la API)
`LICITACIONES_CAMPOS_ADICIONALES`	`fechas,documentos`	Trae el cronograma y los documentos

`Transformacion/.env`

Variable	Valor	Uso
`OPENAI_API_KEY`	clave de OpenAI	Sin clave, se usan solo las reglas
`OPENAI_MODEL`	por defecto `gpt-5-mini`	Modelo de la clasificación
`SAML_USAR_LLM`	`true` / `false`	Apaga el LLM sin borrar la clave

Una variable definida en el entorno del sistema prevalece sobre el `.env`.

6. Ejecución

6.1 Proceso completo (terminal)

```
.venv\Scripts\python Transformacion\ejecutar_proceso.py --actualizar
```

`Ejecutar_SAML.bat` (raíz del repositorio) ejecuta esa misma orden con doble clic y pasa las opciones adicionales que reciba.

Opción	Efecto
(sin opciones)	Modo **demostración**: usa los CSV ya guardados, no consulta el portal
`--actualizar`	Ejecuta primero la ingesta; si falla o no deja exactamente un CSV nuevo, se detiene
`--sin-llm`	Las entidades sin regla se clasifican solo con reglas
`--sin-actualizar-maestros`	No genera las copias de los maestros (solo `Nuevas_…`)
`--maestros DIR`, `--salida DIR`	Cambian las carpetas por defecto (`datos/maestros`, `datos/salida`)

Los maestros se revisan **antes** de consultar el portal. Duración típica: unos 6 minutos con el LLM (la mayor parte es el LLM) y menos de un minuto sin él.

Códigos de salida: `0` correcto · `1` faltan CSV, maestros o configuración · `2` falló la ingesta · `3` falló la verificación (la carpeta de salida se renombra con el sufijo `_REVISAR`).

6.2 Desde n8n

Flujo	Qué ejecuta
SAML - Proceso completo: Licitaciones.info a archivos por asesor	`ejecutar_proceso.py --actualizar`
SAML - Demo con CSV existentes (sin consultar el portal)	`ejecutar_proceso.py`

El nodo **Resultado: archivos por asesor** falla (en rojo) si el código de salida no es 0, si no se generó la revisión o si la verificación no es OK, y devuelve: `modo`, `ingesta_csv_nuevo`, `reparto`, `archivos_por_asesor` (asesor, archivo, ruta, filas), `revision`, `carpeta_salida`, `verificacion` y `csv_consolidados`.

En Docker no hay Excel, así que la ejecución de n8n no genera las copias de los maestros. Después de n8n, en el equipo Windows con Excel:

```
Actualizar_maestros.bat
.venv\Scripts\python Transformacion\actualizar_maestros.py [datos\salida\SAML_<fecha>]
```

Las dos órdenes hacen lo mismo. Sin carpeta, toman la más reciente de `datos/salida`. No repiten una copia ya generada y no actualizan un maestro cuya huella SHA-256 cambió desde que se generó esa salida, para no duplicar obras.

6.3 Etapas por separado

```
cd Ingesta && ..\.venv\Scripts\python extract.py && cd ..
.venv\Scripts\python Transformacion\generar_archivos_asesores.py
.venv\Scripts\python Transformacion\perfil_csv.py
```

`generar_archivos_asesores.py` limpia, reparte y genera los archivos sin verificar ni actualizar maestros. `perfil_csv.py` muestra el perfil de solo lectura de un CSV de la ingesta.

---

7. Lógica del proceso

7.1 Limpieza (`limpieza_licitaciones.py`)

- Lee todos los `Ingesta/output/licitaciones_*.csv` (no los modifica) y deja una fila por `idContrato`; ante repetidos gana el CSV más reciente.
- Solo pasan los procesos `estado_agrupado = Adjudicado`; el resto queda en Incidencias.
- `Valor` (pesos) se convierte a millones; un 0 queda vacío.
- Ganadores sin duplicados idénticos; se marca si cada ganador figura entre los proponentes.
- Del cronograma se toman las fechas de los hitos de adjudicación, de **inicio de ejecución** y de **fin de ejecución** (plazo).
- Nunca se exportan `jwt_contratista_id`, `pivot` ni los campos de estado del usuario.

7.2 Lectura de los maestros (`maestros.py`)

- Una hoja es de obras si tiene `Entidad` en A2, `Valor por Ejecutar` en S2, `Estado de la obra` en BP2 y una fórmula en S3. Así se excluye «Archivo (no vigentes)».
- Valida que 28 columnas clave estén en su letra (`COLUMNAS_ESPERADAS`): si GRECO mueve una columna, la carga falla con un mensaje claro en vez de escribir datos en la columna equivocada.
- Por cada hoja guarda: filas 1 y 2 (encabezados, porcentajes T1:V1, meses X2…), la fila 3 como plantilla (fórmulas y formatos), anchos, categoría de la columna A, número de obras y fila de la última obra.
- El asesor de cada archivo sale de la columna E; el archivo con la hoja «Cobertura clientes» es el de Sugeine.
- Recoge los códigos de las obras existentes (columnas M y AZ de todas las hojas, incluido el archivo de no vigentes) con su empresa y su valor.

7.3 Reparto (`obras_asesores.py`)

1. Duplicados. Un proceso ya está en los maestros si su código, comparado sin espacios ni signos (`FDLBOSA LIC 001-2026-` = `FDLBOSA-LIC-001-2026`), coincide **y** además coincide la empresa (similitud ≥ 0,88) o el valor (±2 %). Si solo coincide el código, se agrega y se anota, porque códigos genéricos como `LP-001-2026` se repiten entre entidades.

2. Entidades con regla (`REGLAS_HOJA`, sobre el nombre normalizado):

Entidad contratante	Hoja	Asesor
Instituto de Desarrollo Urbano, TransMilenio	IDU	Gustavo
EAAB / Acueducto	Acueducto	Mariana
IDRD	IDRD	Asesor 3
Alcaldía Local / Fondo de Desarrollo Local	Alcaldías Locales	Mariana
Alcaldías municipales, ICCU, Fondo de Desarrollo de Proyectos de Cundinamarca, IDUVI (en Cundinamarca)	Municipios Aledaños	Asesor 3

3. Entidades sin regla. Con clave de OpenAI las decide el LLM (sección 7.4). Sin clave, o si el LLM falla para un proceso, se aplica `clasificar_sin_regla`, en este orden: 1. Ubicación sin Bogotá ni Cundinamarca → **Fuera de zona** (descartada). 2. INVIAS / ANI → Municipios Aledaños. 3. Gobernación de Cundinamarca fuera de Bogotá → Municipios Aledaños. 4. Solo mantenimiento, sin adecuación ni obra civil → **Sin consumo de material** (descartada). 5. Obra en Cundinamarca fuera de Bogotá → Municipios Aledaños. 6. En Bogotá, **por tipo de obra**: vial o espacio público → IDU; edificaciones y adecuaciones → IDRD (Asesor 3); parques → IDRD; redes → Acueducto. 7. Lo que no encaja → archivo **Sin asignar (GRECO)**.

Las filas de entidades sin regla llevan el nombre de la entidad en la columna A.

4. Sugeine. Si un ganador es cliente de su hoja «Cobertura clientes» (comparación sin tildes, signos ni sufijos como S.A.S. o LTDA, o confirmación del LLM), la obra se copia a su archivo (las edificaciones, a «Edificaciones y Mant.») y en el archivo del otro asesor queda con `Asesor = Sugeine`. La fórmula de Valor por Ejecutar del maestro la pone en 0 para no contarla dos veces. Si el nombre solo se parece, queda como `<asesor> (posible Sugeine)`.

5. Armado de la fila. Cada fila tiene las 71 columnas A:BS de la hoja de destino:

Columna	Valor
A Entidad	Categoría de la hoja, o el nombre de la entidad si no tiene regla
B Localidad	Localidad de Bogotá tomada del nombre de la entidad, municipio de la ubicación o `Bogotá D.C.`
E Asesor	Asesor de la hoja, `Sugeine` o `<asesor> (posible Sugeine)`
I Empresa	Ganadores separados por `|`
J Notas	`[SAML · fecha] ADJUDICADA · VERIFICAR INICIO` – origen de Inicio/Meses y pendientes

Columna	Valor
K Nombre de la Obra	Objeto del contrato
L Tipo de contrato	Propuesto por palabras clave o por el LLM, de la lista del maestro
M Contrato No · AZ Proceso	Código del proceso
N Valor Millones	Valor de la API / 1.000.000
P Avance · BB Meses de espera	0
Q Meses por Terminar	Del cronograma si es claro; si no, vacío
R Inicio de obra	Mes de inicio del cronograma si es claro; si no, mes de la extracción (nunca antes de X2)
AY · BA · BH	Origen `NUEVA · SAML…`, fecha de cargue, fuente del valor
BI:BM	Datos de Sugeine, si aplica
Fórmulas (S:AX, BC:BG, BN:BS)	Las de la fila 3 del maestro, ajustadas a cada fila
C, D, F, G, H, O, BL	Vacías: las diligencia el asesor

Inicio y Meses desde el cronograma. El cronograma de SECOP guarda versiones anteriores (adendas), así que se compara por **mes**: si todas las fechas de inicio de ejecución caen en el mismo mes, ese es el Inicio de obra; si además hay un único mes de fin, Meses por Terminar = meses desde el inicio (o desde X2, si empezó antes) hasta el fin, entre 1 y 60. En cualquier otro caso Meses queda vacío y se anota.

7.4 Clasificación con LLM (`clasificador_llm.py`)

- **Cuándo:** solo para las entidades sin regla de reparto (unas 100 de cada 140 adjudicadas). Las que tienen regla nunca van al LLM.
- **Envío:** lotes de 20 procesos a la Responses API de OpenAI (`client.responses.parse` con un esquema Pydantic). Por proceso se envían id, entidad, objeto (600 caracteres), ubicación, valor y ganadores. Las instrucciones incluyen las hojas con su descripción y asesor, los tipos de contrato del maestro y la lista de clientes de Sugeine.
- **Respuesta** por proceso:
- `destino`: una hoja de asesor, `Sin consumo de material` o `Fuera de zona`. No existe la opción «sin asignar»: toda obra que consume material en la zona queda con un asesor.
- `tipo_contrato` y `localidad`.
- `consume_material`.
- `cliente_sugeine` (CONFIRMADO, POR CONFIRMAR o NO) y `cliente_sugeine_nombre`.
- `confianza` (alta, media o baja) y `justificacion`.

- **Validación local:** destino y tipo deben estar en las listas del maestro, y el cliente de Sugeine debe existir en «Cobertura clientes». Una coincidencia exacta local con un cliente siempre vale. Lo inválido o faltante se clasifica por reglas.
- **Errores:** clave rechazada (401), modelo inexistente (404), límite de uso, red o servidor → esos procesos van por reglas y se avisa; la ejecución no falla.
- **Trazabilidad:** la justificación queda en la columna de revisión **SAML · Clasificación**; la hoja Procesos marca «Reglas» o «LLM»; el Resumen muestra modelo, tokens y avisos.
- **Para ajustar:** `INSTRUCCIONES` y `DESCRIPCION_HOJAS`.
- **Consumo de referencia:** unos 32.000 tokens de entrada y 38.000 de salida por ejecución con `gpt-5-mini`.

7.5 Archivos generados (`generar_archivos_asesores.py`)

Cada ejecución crea `datos/salida/SAML_<AAAAMMDD_HHMMSS>/` y nunca sobrescribe otra.

- **`Nuevas_<maestro>_<fecha>.xlsx`:**
- hoja **Instrucciones**;
- una hoja por hoja de destino, con las filas 1 y 2 del maestro (totales de la fila 1 ajustados), las filas nuevas con fórmulas y formatos, listas desplegables (E, L, R) y, desde BU, columnas de revisión `SAML · …` (entidad, ubicación, enlace, idContrato, fechas del cronograma, Sugeine, clasificación), que no se pegan en el maestro;
- hoja **Listas**.
- **`SAML_revision_<fecha>.xlsx`:**
- **Resumen**: CSV y maestros con su SHA-256, conteos, LLM, copias de los maestros, reglas y mapeo;
- **Sin clasificar**, **Sin consumo de material**, **Fuera de zona** y **Ya en maestros**;
- **Procesos**: todos los adjudicados, con destino y método;
- **Ganadores** e **Incidencias**.

7.6 Copias actualizadas de los maestros (`actualizar_maestros.py`)

Se hacen con **Excel por automatización COM** (`pywin32`) porque openpyxl borra los gráficos y las validaciones extendidas al guardar. Para cada maestro con obras nuevas:

1. Copia el maestro a `<maestro>_actualizado_<fecha>.xlsx`.
2. Por hoja: - selecciona una sola hoja (algunos maestros se guardaron con hojas agrupadas); - quita el filtro activo si lo hay, y lo avisa; - inserta N filas **encima de la última obra**, para que los rangos fijos del Resumen, Mercado 2027 y Top 15 (p. ej. `I3:I93`) se amplíen solos; - copia en ellas la última obra del propio maestro (fórmulas, formatos, validaciones y formato condicional del mismo libro) y escribe los datos de SAML en las columnas que no son fórmula.
3. Revisa que las fórmulas de esas tres hojas cubran las filas nuevas y avisa si no (pasa en hojas con una sola obra).
4. Recalcula y guarda.

Si no hay Excel (Linux, Docker), se omite y se informa.

7.7 Verificación (`ejecutar_proceso.py`)

Antes de entregar, el orquestador comprueba:

- todos los procesos son `Adjudicado` y no hay `idContrato` repetidos;
- cada fila nueva tiene un Asesor de la lista del maestro y la fórmula de Valor por Ejecutar;
- no hay códigos repetidos por hoja ni obras ya cargadas;
- no hay valores con forma de JWT en ningún archivo;
- los CSV y los maestros no cambiaron (SHA-256 antes y después);
- cada copia actualizada tiene, por hoja, las obras de antes más las nuevas, con sus códigos.

Si algo falla, la carpeta queda como `SAML_<fecha>_REVISAR` y el código de salida es 3.

---

8. Pruebas

```
.venv\Scripts\python -m unittest discover -s Transformacion/tests -v
```

Archivo	Cubre
`test_limpieza.py`	Consolidación, filtro, ganadores, campos vacíos, cronograma
`test_asesores.py`	Reglas por entidad y por tipo de obra, duplicados, Sugeine, Inicio/Meses, tipo de contrato, archivos generados
`test_llm.py`	LLM con un cliente simulado: decisiones, validación, Sugeine, fallos de la API, configuración
`test_actualizar_maestros.py`	Excel real (se omite sin Excel): inserción dentro de los rangos, avisos, modo desde una salida de n8n, maestro cambiado
`test_ejecutar_proceso.py`	Orquestador con ingesta simulada: secretos ocultos, códigos de salida, falta de maestros
`fabrica_maestros.py`	Construye maestros sintéticos con la estructura real

Las pruebas nunca usan los maestros reales ni la clave real de OpenAI: el orquestador se prueba con el LLM y Excel desactivados.

Validación con datos reales (noviembre de 2026): - Ingesta de 138 adjudicadas; 103 clasificadas por el LLM; verificación OK. - Los 4 maestros actualizados conservan sus gráficos, no tienen celdas con error y su Resumen incluye las obras nuevas. Por ejemplo, Mariana pasa de 136 a 153 obras. - Los dos flujos de n8n se probaron en Docker Desktop.

---

9. Seguridad y privacidad

- Credenciales solo en `Ingesta/.env` y `Transformacion/.env`, ignorados por git; la salida del proceso oculta correos, contraseñas, tokens Bearer, JWT y claves `sk-…`.
- Los maestros y las salidas (NIT, ventas, clientes) están en `datos/`, ignorada por git.
- **Datos enviados a OpenAI:** datos públicos de SECOP y la lista de clientes de Sugeine (decisión de GRECO).
- n8n: puerto solo en `127.0.0.1`, código y `.env` montados en solo lectura, maestros en solo lectura. Execute Command está habilitado: no exponer el puerto ni importar flujos de terceros, y detener el contenedor cuando no se use.

---

10. Mantenimiento

Necesidad	Dónde
Cambiar qué entidad va a qué hoja	`REGLAS_HOJA`, `PATRON_MUNICIPAL`, `TIPO_A_HOJA` en `obras_asesores.py`
Cambiar cómo decide el LLM	`INSTRUCCIONES`, `DESCRIPCION_HOJAS`, `HOJAS_EXCLUIDAS` en `clasificador_llm.py`
Cambiar de modelo	`OPENAI_MODEL` en `Transformacion/.env`
Nueva versión de los maestros (VF2…)	Reemplazar los archivos en `datos/maestros/`; si se mueven columnas, ajustar `COLUMNAS_ESPERADAS` en `maestros.py`
Un asesor nuevo	Su maestro en `datos/maestros/` con la misma plantilla; el asesor se lee de la columna E
Tipos de contrato	Hoja «Listas» del maestro (columna A) y `REGLAS_TIPO`

11. Limitaciones conocidas

- La API no trae NIT, número de contrato, fecha real de adjudicación, plazo como campo, coordenadas ni integrantes de consorcios: esas columnas las completa el asesor.
- **Meses por Terminar** casi siempre queda vacío, porque los cronogramas rara vez traen un plazo claro; mientras esté vacío la obra no proyecta en 2027.
- Las copias actualizadas de los maestros requieren Windows con Excel; en Docker se usa `Actualizar_maestros.bat` después.
- En la copia actualizada se quita el filtro activo de una hoja si lo tenía.
- La ingesta trata como «0 resultados» una respuesta de error de la API con cuerpo JSON (distinta de 401): si una ejecución trae 0 procesos sin explicación, revisar la cuenta en Licitaciones.info.
- La clasificación con LLM tarda unos 5 minutos, porque los lotes se envían uno tras otro.